

When Software Security Meets Large Language Models: A Survey

Xiaogang Zhu , *Member, IEEE*, Wei Zhou , Qing-Long Han , *Fellow, IEEE*, Wanlun Ma , *Member, IEEE*, Sheng Wen , *Senior Member, IEEE*, and Yang Xiang , *Fellow, IEEE*

Abstract—Software security poses substantial risks to our society because software has become part of our life. Numerous techniques have been proposed to resolve or mitigate the impact of software security issues. Among them, software testing and analysis are two of the critical methods, which significantly benefit from the advancements in deep learning technologies. Due to the successful use of deep learning in software security, recently, researchers have explored the potential of using large language models (LLMs) in this area. In this paper, we systematically review the results focusing on LLMs in software security. We analyze the topics of fuzzing, unit test, program repair, bug reproduction, data-driven bug detection, and bug triage. We deconstruct these techniques into several stages and analyze how LLMs can be used in the stages. We also discuss the future directions of using LLMs in software security, including the future directions for the existing use of LLMs and extensions from conventional deep learning research.

Index Terms—Large language models (LLMs), software analysis, software security, software testing.

I. INTRODUCTION

SOFTWARE pervades our daily lives, yet its security issues pose substantial risks to many life aspects, such as health, finance, and transportation [1]. For the past decades, several techniques have been proposed and implemented to control the quality of software. These research topics include software requirement that extracts or prioritizes requirements for development [2], software design that intends to provide reliable and fault-tolerant systems [3], software development that estimates the efforts and costs of implementation [4], software testing and analysis that discover defects in programs [5], and software maintenance that enhances software quality after deployment [6]. Among them, software testing and analysis are critical for software security because they are the foremost method to discover defects. The failure of ensuring soft-

ware security may lead to cybercrime, which is projected to cost companies worldwide an estimated \$10.5 trillion annually by 2025, a significant increase from \$3 trillion in 2015, reflecting a year-over-year growth rate of 15 percent [7].

Software testing mainly consists of functional testing and security testing. Functional testing examines the requirements or functionalities from users or customers [8]. Security testing, on the other hand, mainly intends to discover defects and mitigate security problems in software [5]. In this paper, we focus on security testing for software. Based on the knowledge we can obtain for testing, security testing can be categorized into white-box testing, grey-box testing, and black-box testing. The white-box testing knows all information of a target program, such as information of control flow and data flow. On the other hand, the black-box testing knows nothing about the internal information of a target. The grey-box testing has the information of a target between white-box testing and black-box testing. Due to the balance between internal information and engineering efforts, grey-box testing is a widely-used testing method [5]. Fuzzing and unit test are two critical techniques in security testing. Nowadays, fuzzing is a de facto technique in security testing [9]. As to unit test, it is a common practice for developers to control the quality of software development.

Software analysis, especially bug analysis, is another method to mitigate security issues in software. It aims to scrutinize patterns within the code or bugs, with a view to preventing, detecting, or fixing these bugs [10]. For software analysis, research topics include program repair [11], bug reproduction [12], data-driven bug detection [13], and bug triage [14]. Given the substantial manual effort required, researchers continually strive to find solutions that can automate the software analysis.

Traditional deep learning has achieved great success in the fields of natural language process, computer vision, and science [15]. Researchers have also harnessed deep learning to enhance the efficacy of techniques employed in software security [10], [16]. For example, in fuzzing, deep learning improves the quality of highly-structured inputs [17]. In program repair, deep learning automates program repair by directly mapping a buggy code snippet to a benign one [18]. In data-driven bug detection, deep learning automates the feature extraction, which traditionally requires expert knowledge and manual efforts [13]. These instances underscore the considerable potential that deep learning holds for advancements in software security. However, traditional deep learning faces

Manuscript received August 16, 2024; revised September 9, 2024; accepted October 8, 2024. Recommended by Associate Editor Xin Luo. (*Corresponding author: Xiaogang Zhu.*)

Citation: X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.

X. Zhu is with the School of Computer and Mathematical Sciences, the University of Adelaide, Adelaide, South Australia 5005, Australia (e-mail: xiaogang.zhu@adelaide.edu.au).

W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang are with the School of Science, Computing and Engineering Technologies, Swinburne University of Technology, Hawthorn VIC 3122, Australia (e-mail: weizhou@swin.edu.au; qhan@swin.edu.au; wma@swin.edu.au; swen@swin.edu.au; yxiang@swin.edu.au).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/JAS.2024.124971

some challenges, including the difficulty of collecting high-quality training data, the issue of poor generalization ability, and the limitation of understanding complex code. It requires more advanced techniques to solve or mitigate these challenges.

Large language models (LLMs) [19], being a significant breakthrough in the field of deep learning, naturally lend themselves to applications in software security, given the demonstrable success of deep learning in this domain. LLMs can reduce the heavy manual effort required by traditional software security techniques, while also enhancing the effectiveness and accuracy of deep learning applications in software security. Recent research shows that security techniques with LLMs can further be improved even using LLMs in a zero-shot manner [20]–[22]. Therefore, in this paper, we conduct a systematical review of the research employing large language models in the field of software security. We deconstruct each software security technique into several stages in the context of LLM usage. For each stage, we delve into the strategies adopted by the existing studies in the field.

While the existing surveys have shed light on cybersecurity using deep learning [10], [23], cybersecurity using LLMs [24], [25], and software testing using LLMs [26], few of them systematically investigate on software security using LLMs. Some surveys focus on using deep learning to assist software analysis without considering LLMs [10], [23]. Recently, a few surveys review works on using LLMs for cybersecurity. Yet, they focus more on the vulnerabilities and attacks rather than software security [24], [25]. Although a few other surveys analyze how to use LLMs in software engineering, they lack systematic analysis in this direction [26]. Besides, they focus on introducing what software security techniques are, without analyzing steps for using those techniques. This paper contributes to the research on this topic as follows.

1) We systematically review techniques using LLMs in software security. We analyze how LLMs are used in fuzzing, unit test, program repair, bug reproduction, data-driven bug detection, and bug triage.

2) We breakdown each technique into several stages and demonstrate where LLMs are used in each stage.

3) We discuss the future directions of using LLMs in software security. We discuss the potential improvement of existing methods assisted by LLMs. We also discuss the potential to broaden the scope of traditional deep learning in software security.

The rest of this paper is organized as follows. Section II discusses the overview of software security and LLM. Section III analyzes the existing use of LLMs in software testing. Section IV discusses about the techniques of bug analysis using LLMs. Section V explores the potential directions of future works. Section VI concludes this paper.

II. OVERVIEW OF SOFTWARE SECURITY AND LLM

LLMs are not products of chance [27]. In fact, they are the consequential advancements propelled by deep learning breakthroughs. In this section, we briefly introduce the techniques assisted by deep learning in the research area of software security. Then, we introduce basic concepts of LLMs.

We finally introduce how LLMs are used in software security.

A. Software Security by Traditional Deep Learning

Software testing is a widely-used method to mitigate the security issues in software. Since the last century, there have been a wide range of techniques testing the quality of software, such as fault injection testing [28], fuzzing [5], and symbolic execution [29]. To automate the process of testing and release human efforts, deep learning is utilized to speed up the testing. For example, deep learning is utilized to automatically generate highly-structured inputs for fuzzing, which releases the efforts from experts [17], [30]. Deep learning is also used to perform constraint classification for symbolic execution, which largely improves the speed of constraint solving [31].

A more widely used scenario of deep learning is for bug-related analysis, such as bug triage, bug reproduction, bug repair, bug localization, and bug identification [16]. In this scenario, most studies use deep learning to automatically extract patterns in code. Such patterns are then used to achieve tasks of defect analysis [10], [14]. For example, in the task of bug triage, deep learning is used to automatically assign developers to fix a selected bug [14]. In the task of program repair, deep learning is used to automatically generate patches for buggy code [18]. In the task of bug identification, deep learning is used to automatically extract bug patterns, which are further used to detect bugs [13].

The major drawback of using traditional deep learning is that high-quality data is required to be available [16]. For conventional deep learning, data collection and processing are critical to the success of tasks. Due to the convenience of data collection, open-source data is the most popular data used in training deep learning. The open-source resources can be the data collected from GitHub¹, StackOverflow², or CWE³. Yet, the raw data collected from these resources needs to be processed so that they can be fed into deep learning models. For example, code is first translated into graphs (e.g., control flow graph or abstract syntax tree), which are transformed into vectors for deep learning models [10], [16].

B. Large Language Model

Pre-trained language models obtain representations that are effective for diverse tasks [25]. With fine-tuning, such pre-trained models can be used in adapting more specific downstream tasks. Various LLMs have been explored in recent years, such as ChatGPT [32], and Llama [33]. Surprisingly, LLMs additionally present some emergent abilities (i.e., in-context learning, instruction following, and step-by-step reasoning) that conventional models are not capable of [34]. Therefore, LLMs possess a heightened potential on unseen tasks or extrapolating learned knowledge.

For unseen tasks, if no training data about the task is provided, the use of LLMs is called a zero-shot setting. Many studies show that LLMs have the powerful capability of per-

¹ <https://github.com>

² <https://stackoverflow.com>

³ <https://cwe.mitre.org>

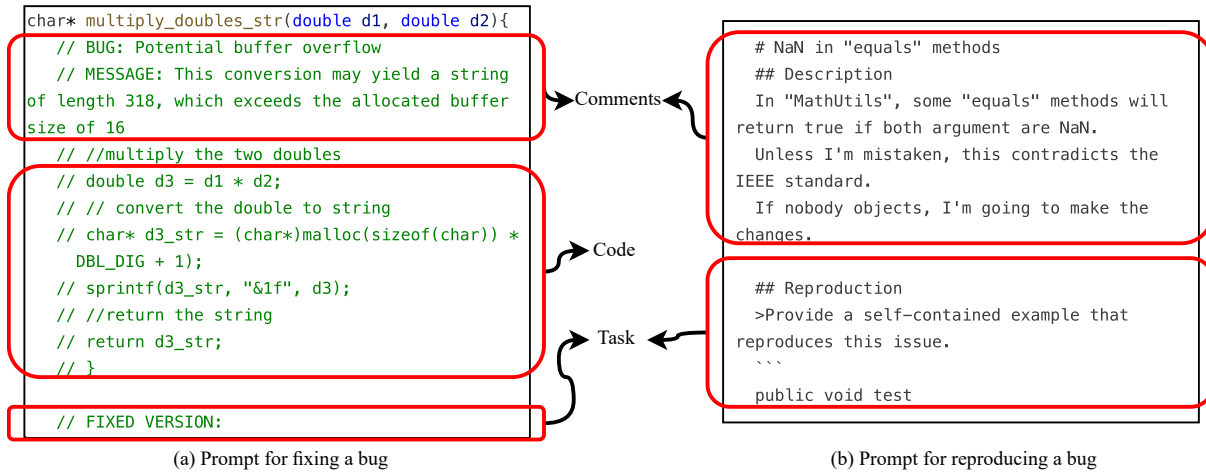


Fig. 1. Two prompts in different scenarios. The prompt for fixing a bug is adapted from [21], and the prompt for reproducing a bug is adapted from [27]. The prompts use comments, code snippets, and task descriptions to query LLM for a required output.

forming zero-shot tasks [19]. This also inspires researchers in software security to use LLMs in a zero-shot way [20], [21]. On the other hand, the few-shot learning provides LLMs with a small size of training data for the downstream tasks. Due to the balance between the computational cost of training LLMs and the performance on tasks, it has a high potential to use few-shot learning in software security tasks.

1) *Pre-Training*: To pre-train an LLM, it is essential to collect large and high-quality training data. The existing LLMs usually pre-train models on texts (e.g., Books1 [35] and Wikipedia⁴) and/or programming code (e.g., GitHub and StackOverflow). Therefore, they have the powerful capability to process various tasks, especially natural language tasks and code generation tasks. Then, the collected data needs to be further processed to be high-quality, clean, and healthy. If an LLM is trained on noisy and toxic data, the performance of it may be decreased.

2) *Fine-Tuning*: Pre-trained LLMs have excellent performance on general tasks, but for more specific tasks, such as bug analysis, high performance often requires model fine-tuning. Adapter tuning and prefix tuning are among the most efficient methods to tune model parameters. The adapter tuning adds small neural networks (the adapters) into the layers of a pre-trained LLM [36]. The prefix tuning prepends some layers to a pre-trained LLM, and freezes parameters of other layers [37]. As later layers are highly related to outputs, another method for fine-tuning is to update or add new layers to the last several layers of LLMs [38], [39].

3) *Prompt Engineering*: Given an LLM, prompt engineering is a lightweight way to use the LLM. A prompt is an input that guides the LLM to generate specific outputs based on the context or instructions provided. By carefully crafting prompts, users can influence the responses of the LLM to achieve desired results without the need for extensive retraining or modifications to the model itself. As for the tasks in software security, many tasks are related to code generation. For example, LLMs can be used to generate function calls for test case generation or to generate code for program repair. As to the

code generation for software security, a prompt may include the information of comments, the related code, and the description of tasks in natural language [21]. An example or an input-output pair will also help LLMs understand a task and generate desired results [19]. Fig. 1 shows two different prompts for fixing a bug and reproducing a bug, respectively. Both of them use comments, code snippets, and task description.

C. Overview of LLM in Software Security

Overall, the existing studies of using LLMs in software security mainly take three steps [21], [40]–[43]. The first step is to collect data for fine-tuning LLMs to achieve a specific task. However, this is not a necessary step in a zero-shot learning. Then, the second step is to perform prompt generation, including the extraction of information for prompts. The third step is post-processing, which checks the results generated by LLMs. For example, researchers check the validity of generated code before the further use of the code. The results of validity can also be used to improve the quality of prompts.

When using LLMs for software security tasks, the existing studies have shown some empirical results. Similar to other tasks, the increase of LLM size improves the performance of software testing and bug analysis tasks, while the computational cost also increases. Most existing works depend on the prompt engineering. A step-by-step prompt generation usually has better performance than a one-step prompt generation. Complex prompt generation does not necessarily perform better than simple prompt generations because too much information in a single prompt may confuse LLMs.

Because the training data of existing LLMs is mostly text and code [19], tasks that are related to text or code generation are explored more in existing studies. For example, in fuzzing, LLMs are used to generate texts for GUI inputs [42] and code for fuzz drivers [20]. For bug reproduction, it mainly uses the ability of text generation of LLMs [27]. For program repair, the ability of code generation paves the way for fixing bugs [44]. However, for tasks such as bug exploit or byte mutation in fuzzing, we have not seen studies exploring on that.

Fig. 2 shows the structure of this paper. Our analysis

⁴ https://en.wikipedia.org/wiki/main_page

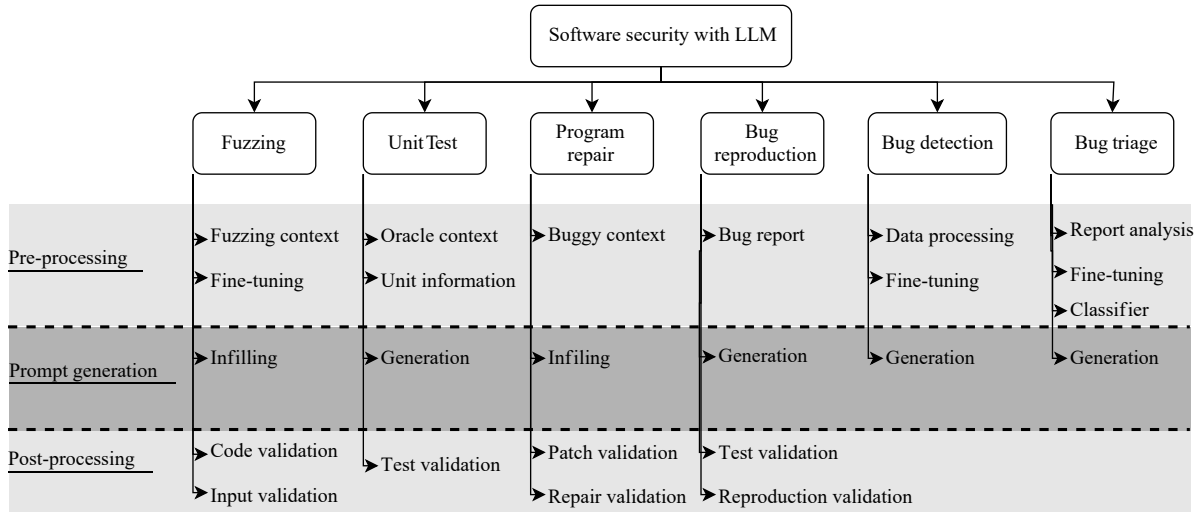


Fig. 2. Software security with LLM. The topics include fuzzing, unit test, program repair, bug reproduction, data-driven bug detection, and bug triage.

includes fuzzing, unit test, program repair, bug reproduction, data-driven bug detection, and bug triage. For fuzzing, unit test, program repair, and bug reproduction, LLM assisted solutions include pre-processing, prompt generation, and post-processing. For bug detection and bug triage, they have pre-processing and prompt generation. The post-processing is not needed possibly because they do not need to validate the generated code.

III. SOFTWARE TESTING ASSISTED BY LLM

Software testing is critical to expose security issues in software by executing tests. It can find defects during the development, before the deployment, and after the release of a software, thus preventing attacks and financial losses. Software testing is traditionally labor-intensive due to the complexity of software, bugs, and execution environments. This motivates researchers and engineers to automate the process of testing.

Usually, software testing includes different phases that are unit testing, module testing, integration testing, acceptance testing, system testing, and regression testing [45]. The unit testing is usually the first phase to be performed, and it evaluates units in an isolated environment. Similarly, the module testing evaluates modules in an isolated environment. The next phase is the integration testing, which assesses a program when integrating different components, such as different units or modules. The acceptance testing intends to meet the requirements from customers, users or business logic when designing a program. The system testing examines the cost, quality, and architecture of a product. Regression testing is performed when a program is changed and it intends to ensure that the change does not harm the existing functionalities [46].

In software testing, the research problems include designing test cases [47], prioritizing test cases [9], optimizing test cases [48], obtaining oracles [49], and synthesizing mutants [50] for a program under test (PUT). Usually, there will be a goal or criterion to design, prioritize, and optimize test cases. For obtaining oracles, the oracles are used to determine the correctness of a test case when the test case is fed into a running program. Finally, synthesizing mutants is achieved by

inserting faults into the original program, which creates related faulty programs, i.e., the mutants. The goal of synthesizing mutants is to mimic the faults that may be introduced by developers, uncovering those faults in advance. While there are various testing phases and research problems, existing works that use LLMs for software testing focus on designing test cases for unit testing and system testing [20], [41], [42]. As shown in Table I, these works utilize the LLMs' ability of code generation or text generation to create test cases. Tasks that include code generation will validate the syntax or compilation correctness of the LLM-generated code.

A. Fuzzing

Fuzzing is a widely-used technique in software testing and has detected lots of bugs in real-world applications [5], [61], [62]. The main idea of fuzzing is to generate numerous test cases for a program. As shown in Fig. 3, fuzzing usually performs fuzz driver construction, input generation, input prioritization, execution environment preparation, input mutation, and quality assessment. These steps mainly build up the execution environment, generate inputs, and assess each execution automatically. The execution quality assessment decides whether the current execution discovers a defect or not. Many fuzzers use crash to indicate a defect while some others use differential testing to determine a defect [5].

Due to the excellent capability of LLMs to generate code and texts, existing fuzzing works mainly use LLMs in a zero-shot manner [20], [41], [42]. The LLMs are utilized to replace the components of constructing fuzz drivers, generating inputs or mutating inputs. Fuzz drivers are testing harnesses that are used to provide executable environment with the targets that are not executable [63]. For example, to invoke functions in libraries of C/C++ codebase, one needs to write a main() function and calls those functions. From the perspective of LLMs, this is a task of code generation. More specifically, since we want to generate fuzz drivers that can cover high code coverage, this is a task of program synthesis, which can generate programs that have specific functionality [64]. As for input generation in fuzzing, the input can be code or text based on target programs. For example, the input for an image

TABLE I
WORKS IN SOFTWARE TESTING BASED ON LLM

Paper	Year	Task	Technique	Base LLM	LLM Usage	Dataset Resource	Validation	Target
[39]	2021	Code Generation	Fuzzing	GPT-2 [51]	Fine-tuning	GitHub	√	JavaScript
[41]	2023	Code Generation	Fuzzing	Codex [52]	Zero-shot	N/A	√	Python
[42]	2023	Text Generation	Fuzzing	GPT-3 [35]	Prompt-tuning	Rico [53], FrontMatter [54]	×	GUI
[20]	2023	Code Generation	Fuzzing	Codex, InCoder [55]	Zero-shot	N/A	√	DL Libraries
[56]	2024	Text Generation	Fuzzing	N/A	Prompt	N/A	√	Protocols
[57]	2022	Code Generation	Unit Test	Codex	Few-shot	Examples	√	Java
[58]	2023	Code Generation	Unit Test	Codex	Zero-shot	N/A	√	JavaScript
[59]	2023	Code Generation	Unit Test	GPT-3.5 [32]	Zero-shot	N/A	√	Corpora
[59]	2023	Code Generation	Unit Test	GPT-3.5	Prompt-tuning	N/A	√	JavaScript
[60]	2023	Code Generation	Unit Test	N/A	Pre-training	Collection	×	Python & Java

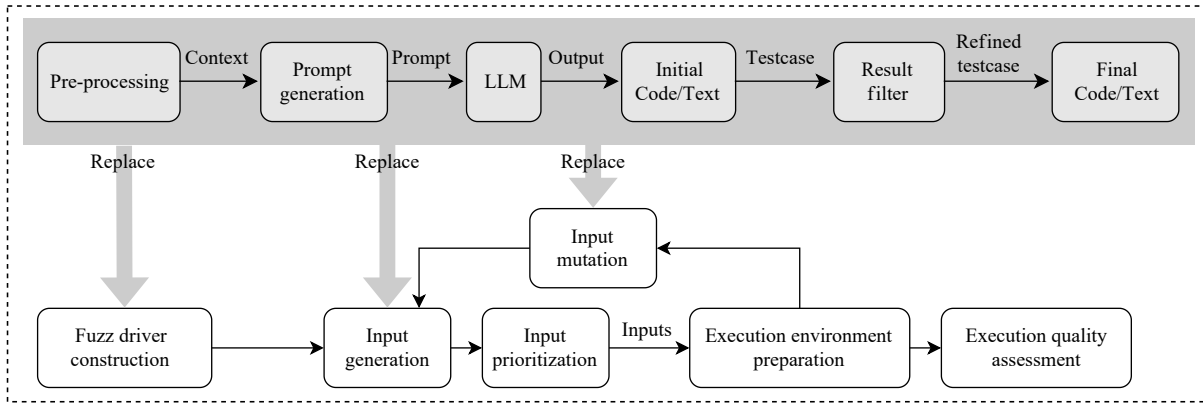


Fig. 3. Fuzzing and its components replaced by LLM in existing works. Existing works focus on replacing the components of constructing fuzz drivers, generating inputs, and mutating inputs by LLM.

processor is an image while the input for JavaScript engine is a code file of JavaScript. Thus, from the perspective of LLMs, the input generation in fuzzing is a task of code or text generation.

The existing workflow of using LLM in fuzzing is shown in Fig. 3. Generally, researchers first process the data required for the LLM task. Then, the results are used to create appropriate prompts for LLMs. Prompt engineering is the key technique for the success of using LLMs [65]. With the help of prompt engineering, LLMs will generate the required data (code or texts). In some cases, the generated data needs to be validated. For example, the generated code needs to be checked if they can be successfully compiled. This step filters out those unsuccessful data and keeps those successful data. Finally, the generated data will be used by fuzzing to improve the effectiveness and efficiency of software testing.

1) *Pre-Processing*: The pre-processing is utilized to extract information for the generation of appropriate prompts. The information is usually the context of the target prompt. For example, to generate prompts that trigger LLMs to generate inputs for graphical user interface (GUI), the context can be the information of texts, images and buttons around the input widget [42]. The context can also be the information of the desired code, such as the target library [20] or module [41] under test. Another context for prompts is the information of

execution states, such as the coverage information [41]. Because a program may have too many code lines that exceed the token limitation of LLMs, such context needs to be trimmed before being used.

2) *Prompt Generation*: With the information for prompts, the next step is to generate prompts. A prompt is a series of statements that provide the information of critical context and output form, and it has unfilled slots for LLMs to complete. As for fuzzing that uses LLMs, the unfilled slots can be the fuzz drivers, the code lines, or the texts that are required for fuzzing. For example, to fuzz functions in a library, the unfilled slots are the code to call those functions [20], [41]. The unfilled slots can also be the inputs required for a target application, such as the texts for a mobile application [42].

3) *Post-Processing*: With the obtained prompts, an LLM will generate code/texts to complete the generation task. Since fuzzing is a dynamic technique, which runs code to test software, the code generated by LLM needs to be validated. The generated code is required to be successfully compiled and executed. For example, in order to test deep learning libraries, LLMs generate code of API calls for fuzzing, and the code is validated that it can be successfully compiled [20].

4) *Integration Into Fuzzing*: With the required code/texts generated by LLMs, the next step is to integrate them into fuzzing. The integration of generated fuzz drivers and initial

inputs are straightforward to fuzzing [56]. Because a fuzz driver is a program that calls application programming interface (API), the harness can be generated by LLMs before running tests. Thus, the integration of LLMs that generate fuzz drivers does not decrease the fuzzing speed. Similarly, the initial inputs for fuzzing also do not decrease the execution speed. Yet, when the input generation interacts with other fuzzing steps, the generation speed of LLMs will influence the efficiency of fuzzing. Input mutation is a process that mutates the initial inputs and get more new inputs. Therefore, the integration of LLM and input mutation might significantly decrease the execution speed of fuzzing, especially when the input mutation by LLM has to perform pre-processing.

5) *Fine-Tuning*: Although the existing fuzzers mainly use LLMs in a zero-shot manner, fine-tuning LLMs is still an option to improve fuzzing efficacy [39]. For example, in order to test JavaScript engines, LLMs can be used to generate JavaScript code files. To further improve the performance of such LLMs, one can fine-tune a pre-trained LLM. The challenge is to collect data for the task [39]. To fine-tune a pre-trained model on JavaScript code, one can collect data from GitHub. Another challenge of fine-tuning LLMs is the requirement of enormous computing resources. Thus, researchers can work on slighter ways to fine-tune a pre-trained LLM.

6) *Comparative Analysis and Limitations*: While existing LLM-assisted fuzzers follow similar process to perform fuzzing, they aim at different testing scenarios, which include Python programs [41], protocols [56], mobile GUI [42], deep learning libraries [20], and JavaScript Engines [39]. As shown in Table I, the different testing targets result in different use of LLMs. For instance, to test protocols and GUIs, they use LLM to generate inputs, which are in the form of texts. To test deep learning libraries and JavaScript Engines, they use LLMs to generate programs to invoke APIs and test compilers, respectively. Moreover, the existing fuzzers also use LLMs in a different way. As shown in Table I, fine-tuning, prompt tuning, and zero-shot prompt engineering are the main different ways to use LLMs.

Although the existing fuzzers have achieved great success in using LLMs, they are still challenged by the complexity of fuzzing processes. For example, the existing fuzzers still need to manually confirm bugs, which requires much effort to analyze bugs. For fuzzers that use prompt engineering, especially for the task of code generation, LLMs may suffer from false positives of generating executable code. In most situations, LLMs are further used to fix issues in generated code. For solutions that fine-tune the existing LLMs, they suffer from data bias or data paucity in the training set.

7) *Empirical Takeaway*: Employing LLMs on fuzzing improves the performance of fuzzing, including code coverage and bug discovery, even in a zero-shot manner. The temperature setting of LLMs can influence the performance of fuzzing, and empirically, a higher temperature positively increases the code coverage found by fuzzing. As for the prompts, more complex prompting does not necessarily generate better test cases for fuzzing. However, applying LLMs with longer time or generating prompts with step-by-step instructions usually improves the fuzzing performance.

Finally, it costs more time for LLMs to help generate or mutate test cases than the native operations of fuzzers. Since execution speed is critical to fuzzing [5], future adoption of LLMs can improve the speed of test case generation.

B. Unit Test

Unit test is a common practice, where developers write test cases together with a unit to dynamically check the quality of a unit. A unit can be a method, a class, or a function that can be logically isolated from a system. Similar to fuzz drivers for APIs in libraries, unit test also needs to write code (e.g., testing harness) to run tests for a unit. Different from testing libraries, unit test usually has a test oracle to determine if execution results are the expected behaviors, mostly by assertions [66]. Therefore, when using LLMs for unit tests, the generated code by LLM needs to contain extra code for determining the correctness of a unit.

It is labor-intensive to manually generate unit tests in a project. Thus, many research works focus on the automatic generation of unit tests [67]. Some automatic generation tools for unit testing, such as EvoSuite [68] and Randoop [69], are developed to mitigate the efforts of developing unit tests. However, evaluations on those tools reveal that the generated unit tests cannot expose defects that require specific primitive values or complex configuration [70]. Some studies even show that the use of automatic generation framework even decreases the detection rate of defects while experiencing an increase of development time [67]. Therefore, automatic generation tools require to improve their quality of unit tests.

As shown in Fig. 4, similar to fuzzing, the LLM-assisted unit test includes pre-processing, prompt generation, and post-processing [57]–[59]. The generation process generally conforms to the generation-validation-repair steps. Some other works pre-train an LLM to generate unit tests directly [60]. To automate the unit testing, the documents for the target program are first learned to extract information of target units and the corresponding descriptions. With such information, appropriate prompts are generated so that unit test code can be further generated by LLMs. While the generated unit test may have problems such as the compiling issues or ineffective assertions, researchers refine the code in post-processing.

1) *Pre-Processing*: The information for generating appropriate prompts can be obtained by analyzing project code files and documentation. This is to get the information of the target units and their assertions. In this step, the pre-processing can get the information of APIs and their related access path [59], comments and functionality descriptions [59], and other unit related information such as class signature, method signature and method invocation [58]. This may include control flow analysis [71], data flow analysis [72] or abstract syntax tree (AST) analysis [73].

2) *Prompt Generation*: The result of pre-processing may produce prompts with a large size, i.e., too many tokens. Therefore, it is critical to trim the token size when generating prompts. One solution is to generate prompts based on the importance of context, i.e., highly important contexts are included in the prompt first [58]. Another solution is to generate a simple initial prompt first, and gradually refine the

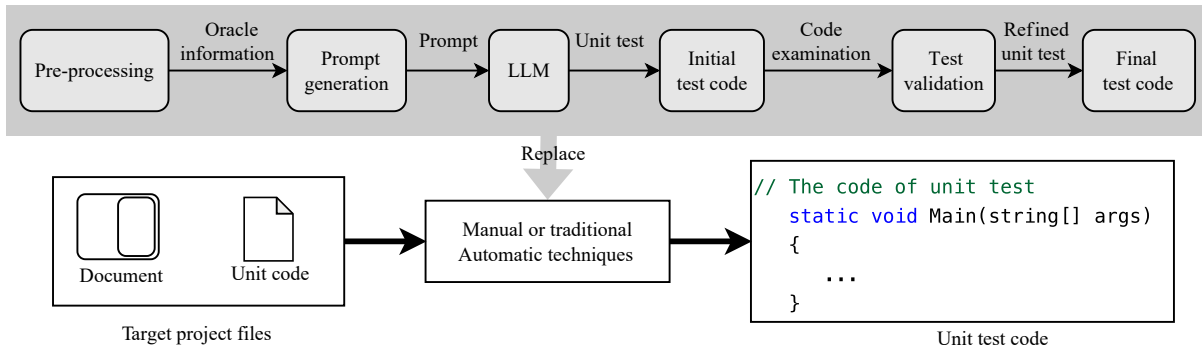


Fig. 4. Unit test assisted by LLMs. An LLM generates unit test code based on the prompt extracted from the document and target projects. The resource for generating unit tests includes the documentation of target project and code of target unit. The result is the code for unit test.

prompt by adding more information [59]. The generation of prompts can also be conducted in an interactive way, where previously generated prompts and tests can be the input to further generate better prompts and tests [59]. As a result, a prompt will contain unit signature, documentation comment, use examples, or source code for a unit under test.

3) *Post-Processing*: The post-processing intends to validate the generated unit tests. This will check the compiling issues, syntactic correctness, and simple dependency issues. For example, the existing works may compile the generated code to check if any error occurs, or check if the code misses brackets [58], [59], and then repair simple errors based on some patterns. If the problems cannot be fixed, they will use LLMs to fix those errors. For example, they will extract related error information and context to generate prompts for LLMs, which may generate correct unit tests.

4) *Integration Into Fuzzing*: While none of the existing works use LLMs to integrate unit test with fuzzing, it is possible to achieve the integration. A study has already used unit tests to generate fuzz drivers by inferring API dependencies [66]. Alternatively, after the generation of a unit test, LLMs can be utilized to generate inputs for the test. For example, in Fig. 4, fuzzing can generate values of integer for the method Calc(). The challenge is to generate inputs that can satisfy the syntactic and semantic correctness of a unit, as well as the high code coverage. This can be achieved by LLMs as demonstrated by existing works in Section III-A.

5) *Comparative Analysis and Limitations*: For generating unit tests, an essential challenge is to generate effective oracles. While the existing LLM-assisted solutions follow the general steps, they generate the oracles based on different resources. One way to extract test oracles is to analyze the natural language documentation of the target project [57]. In this scenario, the approach of using LLMs is to query responses from them in a few-shot way. A similar but slightly different way is to generate oracles by parsing target projects, including both code and documentation [58]. This is achieved by creating an adaptive focal context that leads to effective unit test generation. To further improve the quality of unit test generation, researchers analyze the target unit before querying LLMs [59]. As a result, the information contained in prompts includes function signature, documentation comments, usage examples, and source code. The rich information provides

LLMs with more insights to generate effective unit test code. Another different way is to train a customized LLM, which utilizes a signal to map between code and test files, to generate unit tests for Python and Java programs [60]. Because this LLM is specific for unit test, it achieves great success in generating unit test code.

The existing solutions try to use LLMs to generate unit tests in less number of inquiries (zero- or few-shot learning). They directly query LLMs without complex analysis of target programs. For example, when the task is to understand the functionality of target programs, existing solutions rely on comments/documentation [57], [59] or normal program analysis such as extracting Abstract Syntax Tree [58]. This leads to a high percentage of generating code that is not executable [58]. Analyzing target programs to extract semantic information may be the mitigation to generate executable unit tests.

6) *Empirical Takeaway*: Oracles are critical in unit tests and LLMs can successfully generate the oracles (assertions) for testing. Only a small portion of unit tests generated by LLMs encounter syntax errors; however, compilation errors are more frequent, with approximately one in three tests being affected. Surprisingly, LLMs do not frequently generate same or similar tests. As for prompt generation, simple and accurate description of tasks in natural language helps improve the quality of generated tests. It also improves the quality of unit tests if LLMs are provided with suitable examples. Adding more information into prompts may confuse LLMs, and the confusion results in generating failing tests. Finally, while increasing the model size of LLMs improves the performance of unit tests, the computational cost also increases.

IV. BUG ANALYSIS ASSISTED BY LLM

Machine learning has been employed for bug analysis for decades across the fields of bug repair, bug reproduction, bug identification, and bug triage. The common challenge for machine learning is to construct features for bugs, which usually requires expert knowledge. Deep learning can automatically capture the features, which releases the manual efforts, but still suffers from issues, such as data imbalance, over-fitting, or under-fitting [10]. LLMs have the potential to overcome those problems, and recent works have explored the possibility of using LLMs in bug analysis. As shown in Table II, LLMs have been used in bug repair, reproduction,

TABLE II
WORKS IN BUG ANALYSIS BASED ON LLM

Paper	Year	Task	Topic	Base LLM	LLM Usage	Aim	Validation
[74]	2021	Code Generation	Repair	Codex	Zero-shot	Study	√
[22]	2022	Code Generation	Repair	CodeBERT [75]	Zero-shot	New Tool	√
[76]	2022	Code Generation	Repair	Codex	Zero-shot	New Tool	√
[77]	2022	Code Generation	Repair	CodeT5	Fine-tuning	New Tool	×
[78]	2023	Code Generation	Repair	GPT-3.5	Zero-shot	New Tool	√
[44]	2023	Code Generation	Repair	GPT, Codex, CodeT5 [79], InCoder	Zero-shot	Study	√
[80]	2023	Code Generation	Repair	Codex	Zero-shot	Study	√
[21]	2023	Code Generation	Repair	Codex, Jurassic [81], polycoder [82]	Zero-shot	Study	√
[43]	2023	Code Generation	Repair	PLBART [83], CodeT5, CodeGen [84], InCoder	Zero-shot/ Fine-tuning	Study	√
[85]	2023	Code Generation	Repair	PLBART, CodeT5, CodeGen, InCoder, Codex	Zero-shot/ Fine-tuning	Study	√
[86]	2023	Code Generation	Repair	Codex	Few-shot	New Tool	×
[87]	2023	Code Generation	Repair	CodeBERT, GraphCodeBERT [88], PLBART, CodeT5, UniXcoder	Fine-tuning	Study	√
[89]	2023	Code Generation	Repair	CodeT5, InCoder	Predicting	New Tool	×
[27]	2023	Code Generation	Reproduction	Codex	Zero-shot	New Tool	√
[40]	2022	Transfer Learning	Detection	Megatron-LM [90], GPT-2, CodeBERT, RoBERTa [91]	Fine-tuning	Study	×
[92]	2022	Text Generation	Detection	InstructGPT [93], OPT [94]	Zero-shot	Study	×
[95]	2023	Text Generation	Detection	GPT-3	Zero-shot	Study	×
[38]	2022	Transfer Learning	Triage	RoBERTa	Fine-tuning	New Tool	×

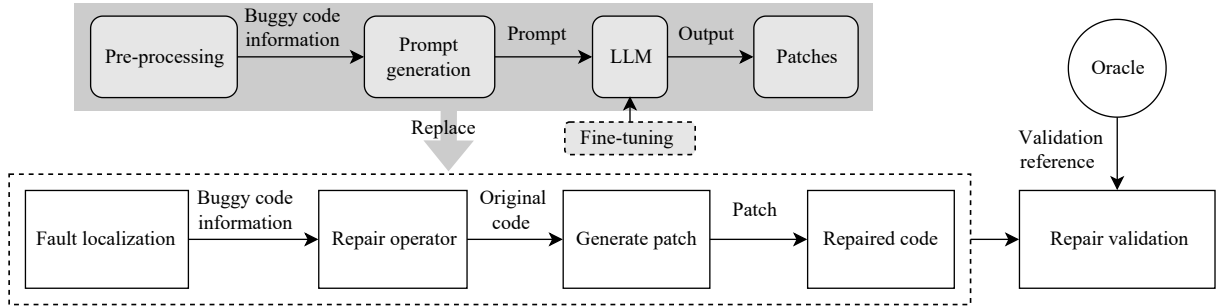


Fig. 5. Automatic program repair assisted by LLM. With the bug information, LLM is utilized to generate patches for target programs. Validation is then performed to verify that the program is successfully repaired.

detection, and triage. Existing works use a variety of base LLMs to achieve their tasks. When LLMs are used to generate code for other downstream tasks, the generated code is often validated to check if the code can be compiled or run correctly. Many of these studies investigate the potential of using LLMs in bug analysis while some others propose a new tool in this area.

A. Automatic Program Repair

Automatic program repair (APR) intends to fix unexpected program behaviors (i.e., bugs) in an automatic manner either statically or dynamically [96], [97]. The program repair includes behavioral repair and state repair [96]. The behavioral repair changes the code of a program so that it fixes the behavior of the program [11]. The state repair changes the runtime states to fix programs. The term *state* in this context can be any execution information at runtime, such as the heap, stack, or even environment. To automatically repair a pro-

gram, as shown in Fig. 5, it usually includes steps of 1) localizing the faulty code regions, 2) applying repair operators to generate patches, 3) repairing the target program, and 4) validating the repair. Usually, APR tools use existing tools to localize the faulty code, such as the statistical fault localization techniques [98]. In order to validate whether the repair is successful, it usually requires an oracle that has the expected behavior of the program.

The traditional APR tools usually need multiple repair actions to correctly fix a bug [97]. To improve the performance of APR, neural machine translation (NMT) is proposed to generate fixes [18]. Motivated by the translation of natural languages, NMT views the task of code repair as a translation problem where the goal is to translate a buggy code into a patched code. The problem is the dataset for training the model, which is labor-intensive. They may need to construct a large number of high-quality samples for training an effective NMT model. Due to the excellent performance of code gener-

ation of LLM, APR can integrate LLM in a zero-shot manner to perform code repair [76]. Some works fuse LLMs with completion engines to generate valid repairs [89].

We conclude the APR via LLMs into three steps that are pre-processing, prompt generation, and post-processing. The pre-processing is performed to localize the buggy lines and analyze the information used for prompt generation. The prompt generation prepares the prompts to inquire LLMs, and the post-processing validates the successful fix of code.

1) *Pre-Processing*: To repair programs, existing APR tools assisted by LLMs require the availability of information about buggy code lines. Note that, it is possible to simply ask LLMs to repair programs without any information of buggy lines. For example, one can simply give the instruction as “Please repair bugs in the program”, and LLMs can sometimes successfully generate patches to repair the program [80].

The existing works get the information of buggy lines from three resources. The first resource comes from the fault localization techniques [22], [80]. Because fault localization techniques usually output the rankings of lines that are probably buggy, LLMs need to generate multiple patches to successfully repair programs [99]. For some other works, researchers curate datasets where buggy lines are known [21], [74], [100]. They may collect data from real-world programs or from websites that are related to bugs [74]. They may also synthesize a dataset of those buggy programs [21]. For example, QuixBugs [101] is a dataset of buggy programs, and CWE [102] is a website that lists weaknesses of software and hardware. In some other settings, researchers just presume that the buggy lines are known [44], [78]. This is reasonable in the scenario of APR because program repair aims to fix bugs.

2) *Prompt Generation*: For automatic program repair, a prompt usually includes code or comments related to the buggy lines. A naive prompt can simply ask an LLM to fix bugs in a program [78], [80]. For a better performance of program repair, a prompt will include information of buggy code lines. For example, a prompt can ask LLMs to fix a specific line such as a line number “Fix line 7” or a statement “Fix $i = -1$ ” [74], [80], [86]. For a more general setting, a prompt includes code that removes the buggy lines [22], [44], [74]. That is, researchers treat the task as an infilling task. The code lines before and after the buggy lines are correct, and LLMs are used to generate patches by replacing the buggy lines [21], [100]. For the comments in a prompt, they can be the hint of buggy lines such as “These are buggy lines”. They can also be the descriptions about the correct program behaviors. Some prompts also include the input-output pairs provided by developers. While program repair may fail at the first repair action, the prompt generation will also include information of the previous repair actions. For example, a prompt can include source code as well as the results of previous validation [78].

3) *Post-Processing*: Similar to LLM-assisted unit test generation, APR with LLMs also needs to check if the generated code can be correctly compiled. Different from unit test generation, APR with LLMs needs to validate whether the repair is successful. Most existing works use oracles to validate a repair [21], [44]. They feed a repaired program with specific inputs, and compare the results to expected program behav-

iors. If programs behave correctly, the repair is successful. Otherwise, it fails. However, this method of validation may generate plausible patches, which can pass when feeding with prepared inputs. They may still fail on other inputs. Another solution to validate the repair result is to use other techniques to verify the repaired code [78]. For example, Bounded Model Checking [103] can be used to verify the code generated by LLMs.

4) *Fine-Tuning*: Intuitively, fine-tuning an existing LLM will result in better performance of bug repair [43], [76], [87]. In the context of bug repair, the dataset for fine-tuning is the pair of buggy code and fixed code. For example, the dataset can be collected from GitHub, where many projects have reports of bugs and patches. The evaluation of fine-tuning demonstrates that it can provide significant improvement in bug repair (46% ~ 164% more fixed bugs) [43], [85].

5) *Comparative Analysis and Limitations*: Many works focus on exploring the potential of using LLMs in software repair, while others focus on developing new tools for program repair, as shown in Table II. Some research investigates how to directly leverage LLMs to fix defects, using generative and infilling approaches [43], [44], [80]. The generative approach generates entire patches, whereas the infilling approach removes buggy code lines and uses LLMs to fill the removed lines. When existing works examine the potential of LLMs in program repair, they often rely on zero-shot learning methods. Conversely, other researchers have developed new tools to fix bugs [76]–[78]. Due to the limited performance of zero-shot methods, tool development efforts often incorporate fine-tuning and few-shot learning techniques [77], [86], both of which tend to outperform zero-shot approaches. An alternative approach to enhancing LLM-assisted program repair is to leverage the power of a Completion Engine, which prunes infeasible tokens and proactively completes tokens during the generation process [89].

The use of LLMs in program repair shows great promise; however, they still face several limitations. One key factor is the significant influence of prompts on the performance of bug fixes. Although LLMs outperform traditional APR tools in some tasks, the success rate of LLM-assisted methods remains relatively low, even when fixing short and synthetic programs. Another critical factor is code size, which can have a substantial impact on the quality of repairs generated by LLMs. Complex bugs often span different regions of the code, leading to a desire to include the entire codebase in the prompt. However, this approach can exceed the token limit of LLMs, restricting the ability to effectively analyze and repair larger programs. Moreover, the size of the LLM itself plays a significant role in program repair performance. Larger models typically yield better results, as they have a greater capacity to understand and generate accurate code repairs. However, this comes with trade-offs in terms of computational resources and inference time, which may not be feasible in all scenarios.

6) *Empirical Takeaway*: The complexity of both bugs and fixes significantly impacts program repair. LLMs tend to perform well when the fixes are short and localized, meaning they only need to generate small patches. It has been observed

that the lines of code following the buggy locations can have a greater influence on the bug fix outcome. LLMs demonstrate varying levels of performance depending on the target applications. For instance, they achieve higher success rates when fixing synthetic programs, which are typically shorter, compared to real-world programs. The success rate is even lower when repairing hardware description languages like Verilog [104], compared to software programs written in languages like C or Python. Additionally, interactions between LLMs and the repair process can improve repair quality. It has been suggested to generate multiple samples and to allow LLMs more time to work on the repair. In conclusion, the success of LLM-assisted program repair depends on factors including the size of the code, the size of the LLMs, the complexity of the bugs, the complexity of the fixes, and the time allocated for LLMs to perform the repairs.

B. Bug Reproduction

Software development will almost inevitably introduce faults into software due to the expensive and limited verification techniques [105]. When users experience software failures, they may report those failures to developers. The failures might be a kind of bug that developers need to eliminate. To achieve that, an important step is to reproduce the reported bugs [12], [106]. Based on the timing of reproducing bugs, the techniques for reproducing bugs can be categorized into record-replay solutions and post-deployed solutions [106]. The record-replay solutions reproduce bugs at runtime by monitoring executions, which is achieved by software instrumentation or hardware recording techniques [107]. The post-deployed solutions reproduce bugs from reports for the deployed software [108]. Because the behaviors of deployed and in-house software are different, these solutions may synthesize executions to reproduce bugs. However, these works only reproduce bugs that crash, which are only a small part of bugs [27].

As shown in Fig. 6, to perform bug reproduction, it usually contains bug report, bug report analysis, test case generation, and reproduction validation. The bug report can come from the users or the testers. With the bug reports, software engineers analyze the reports and generate test cases to reproduce bugs. To validate whether a bug is successfully reproduced, one solution is to validate it based on information of crashes, such as the crash stacks. Another solution is to compare the execution results to the bug reports.

LLMs can be utilized to generate test cases from bug reports

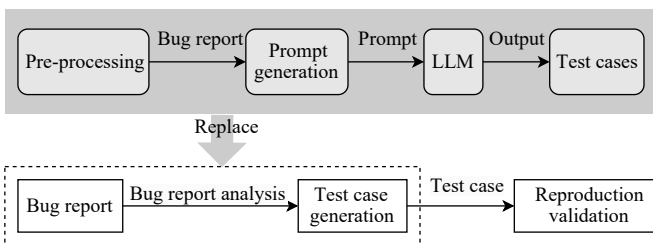


Fig. 6. Bug reproduction. Bug reproduction usually generates test cases based on bug reports. Those test cases are further verified whether they successfully trigger the reported bugs.

due to the impressive performance of code understanding and text generation [27]. The prompt is generated based on the bug report. For example, the bug report can be transformed into a Markdown document; then, prompts will be generated based on the Markdown document [27]. With the prompts, one can use them to query LLMs, which can generate test cases. Such test cases may not be executable, and they are inserted into existing test harness [27]. Finally, to validate if a bug is successfully reproduced, the behaviors of executions are compared to the bug reports. The bug reproduction needs to produce behaviors the same as bug reports. The limitation of LLM-assisted solutions is that the success rate is low. This is because to reproduce a bug, the generated test cases must examine the target code regions, as well as the required execution states. LLMs may miss the information of either target code regions or execution states.

The empirical takeaway from the existing work [27] is that LLM-assisted bug reproduction can successfully reproduce a large number of diverse bugs. Besides, by applying a longer time to generate more test cases, it can reproduce more bugs. Finally, LLM-assisted bug reproduction can reproduce recent bugs, indicating that LLMs indeed predict the test cases rather than simply copy tests.

C. Data-Driven Bug Detection

Data-driven bug detection usually uses machine learning or deep learning to determine if bugs exist or to pinpoint the bug locations. In the academic field of software engineering that uses deep learning, about 20% of works focus on bug identification [16]. The most important research in this field is to design or obtain appropriate features for bugs, which basically indicate the patterns of bugs [109]. For traditional machine learning, the features are designed by experts. The significant drawbacks of feature design are time-consuming and error-prone [10]. To solve this problem, deep learning is utilized to automatically extract the features.

Different types of inputs are utilized for deep learning models to obtain features of bugs, including graph-based inputs, sequence-based inputs, text-based inputs, and mixed inputs [10]. Most of the works use graph-based inputs to train deep learning models, and the graphs can be control flow graphs (CFG), data flow graphs (DFG), abstract syntax trees (AST), or mixed graphs. The sequences in sequence-based inputs include execution traces or sequences transformed from the graphs mentioned before [13], [110]. The text-based inputs mainly regard the program code as inputs for deep learning. The program code can be the source code, assembly code, or their related forms [111], [112].

1) *Fine-Tuning*: For conventional deep learning, the key to successful bug detection is to learn the appropriate features. By applying transfer learning, deep learning models in a domain can be utilized in another domain. Similarly, LLMs already learn features or representations in texts or code, and they can be fine-tuned for tasks in other domains. As shown in Fig. 7, to fine-tune LLMs, the necessary requirement is the data prepared for the specific domain. Then, the raw data is usually required to be processed so that the data can be utilized to fine-tune a pre-trained LLM.

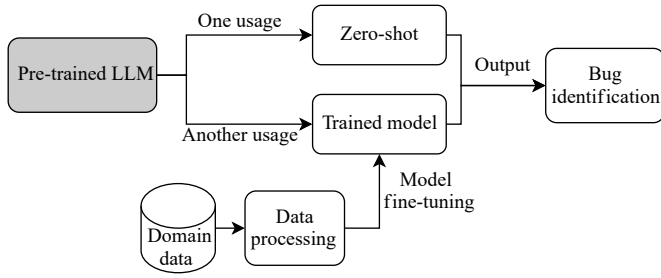


Fig. 7. Data-driven bug detection. LLMs can be utilized to detect bugs in a zero-shot manner or by fine-tuning. Empirically, the use of zero-shot learning in LLMs detects bugs with low accuracy.

To fine-tune a pre-trained LLM for bug detection, the data needs to be related to bugs. In the context of bug detection, the basic types of data are the buggy code and the benign code. For example, to detect bugs in software written in C/C++ programming languages, a dataset of bugs in C/C++ [13] that can be used to fine-tune LLMs is the dataset from VulDeePecker [40]. The more detailed information is the types of bugs. For example, in the VulDeePecker dataset, there are buffer-related bugs and resource-management-related bugs.

Before data can be used to train LLMs, the data needs to be processed so that the items in the dataset are not duplicated and conflicted [40]. In a dataset, it is possible that the same item may have two different labels. The clean data is then used to train an LLM so that the LLM can correctly classify the (types of) buggy or non-buggy code. The next step is to fine-tune an LLM, and one way is to add model layers on top of the LLM.

Interestingly, a larger pre-trained model is not necessarily to produce a model that performs better. The performance of the fine-tuned model possibly depends on the dataset used to retrain LLMs. That is, a larger model requires a larger size of high-quality dataset. Besides, the performance might also be influenced by the hyperparameters such as learning rate. As for the training time, a larger model usually takes longer time to be fine-tuned, taking days to complete fine-tuning.

2) *Zero-Shot Usage*: As shown in Fig. 7, an alternative use of LLM in bug detection is to use it in a zero-shot manner. By directly feeding a code snippet into an LLM, the LLM will determine if the snippet is buggy or the type of bug contained in the snippet. Empirically, LLMs do not perform well on differentiating buggy code from non-buggy code in a zero-shot manner [95]. Another zero-shot use of LLMs is to detect event bugs in video games [92]. The event bugs are not detected in the form of videos or figures, but in the form of texts. By describing the events in a game, an LLM is used to determine if the described events are buggy [92]. For video game bugs, they are mainly logic bugs such as the abnormal animation of a character's body. Similar to bugs of C/C++, the evaluation results show that the accuracy of detecting event bugs is low (about 60%) if using LLMs in a zero-shot manner.

3) *Comparative Analysis and Limitations*: Different targets lead to different usage of LLMs in bug detection. To detect logic bugs in video games, LLMs for natural languages are utilized because videos will be described in natural language [92]. Thus, InstructGPT and OPT are selected to run experi-

ments. To detect bugs in code lines, fine-tuning LLMs is preferred so that training data about code bugs can be learned by LLMs [40]. An alternative way to use LLMs in detecting code bugs is to perform zero-shot learning, which directly queries LLMs to determine if given code is vulnerable [95]. However, the result does not show significant improvement compared to traditional deep learning methods, indicating that zero-shot learning for bug detection is not promising. This may lie in the fact that it has subtle differences between buggy and non-buggy code. This limits the use of zero-shot learning in bug detection.

D. Bug Triage

When a bug is encountered, users or testers often report the bug with a detailed description of the bug. Bug triage is the process of assigning developers to fix bugs that are prioritized based on metrics such as severity or frequency [113]. Due to the large number of bug reports, it is labor-intensive and fault-prone to manually assign a bug to an appropriate developer who is able to fix the bug. To automate the process of bug triage, machine learning is a widely-used technique to achieve the task [113]. Usually, the task of bug triage is formulated as a classification problem, where developers are mapped to a selected bug [14]. Similar to other tasks that use machine learning, a critical step is to extract and select suitable features to achieve the classification of developers. Similar to other tasks, machine-learning-based bug triage is also challenged by constructing appropriate features. Therefore, deep learning is utilized to automatically extract features and achieve bug triage [114].

Recently, LLMs have been utilized for bug triage. An existing work by Lee *et al.* proposes a lightweight framework to achieve bug triage [38]. It compresses an LLM based on a knowledge distillation technique. The knowledge distillation technique helps reduce the size of a deep learning model while maintaining the performance of the model [115]. As shown in Fig. 8, to fine-tune the LLM, it uses the dataset that consists of bug reports and results of triage. Further, it uses convolutional neural networks (CNN) to extract features from the compressed LLM. The CNN-based model performs the task of classifying developers for a selected bug. Basically, this work leverages the representations learned by LLMs, and the representations are further used to achieve the task of classification of developers.

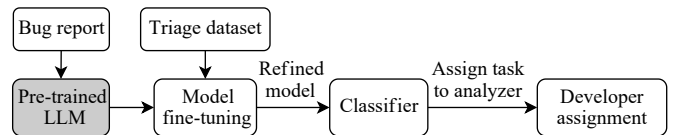


Fig. 8. Bug triage via LLM. The existing work adds a classifier to the later layers of an LLM.

V. FUTURE DIRECTIONS

We discuss the future directions in two aspects. First, we discuss the improvement of the existing use of LLMs in software security. Then, we discuss the possible use of LLMs by borrowing ideas from traditional deep learning.

A. Future Directions for Existing Use of LLMs

While the existing research shows great potential of using LLMs in software security, some challenges still need to be explored and resolved.

1) *For LLMs*: Since training LLMs requires enormous computational resources, it prevents many research groups from training their own LLMs for specific tasks. How can we reduce the training cost of LLMs while maintaining the performance? Most existing works focus on the problems that can be translated into code or text generation. Can LLMs be used in other ways? If training cost can be reduced, can LLMs be integrated tightly into tasks of software security instead of using them in a zero-shot or few-shot manner? For software security, can other large deep learning models be used in this field, such as the large computer vision models? For example, when testing binary code, can we regard binary code as an image, and test them with the help of large computer vision models? We may borrow ideas from the field of object detection [116] or image forgery detection [117], which use deep learning to detect small differences. The small differences in the context of software security can be the bugs in binaries.

2) *Data Augmentation*: As shown in previous sections, to fine-tune an LLM for a downstream task that requires domain data, a major challenge is the training data. A dataset lacking quality or quantity will impair the performance of fine-tuned LLMs. For example, existing studies show that larger models usually have better performance [19]. Yet, larger models usually require a larger amount of data to train them. Large models may perform worse than small ones if they are fine-tuned on a small data size. A potential solution is to use LLM to augment training data for a specific task. Some studies have explored the research on augmenting data using LLMs in different tasks, such as text classification [118], information retrieval [119], and intent classification [120]. Future works may focus on augmenting training data for software security tasks, such as software testing, bug detection, or bug exploitation.

3) *Improvement of Success Rate*: As for tasks depending on code generation, they often validate the LLM-generated code. They check if the code can be successfully compiled or executed. However, LLMs still struggle with generating such code with a high success rate. Future works might focus on improving the syntactic and semantic correctness of generated code. Furthermore, tasks such as program repair or bug reproduction require extra stages to validate if their tasks are successful. For example, program repair needs to check if a program is successfully fixed, and bug reproduction needs to check if a bug is successfully triggered. Can LLMs help with those extra validation stages? For example, can LLMs help ensure that a patch really fixes a program rather than being a plausible patch? Can LLMs help ensure that a bug is successfully reproduced when the bug does not crash?

4) *Less Expert Knowledge*: The existing works show that when generating prompts, it still requires extra stages to extract knowledge from some resources. For example, to fix a bug, a tester needs to first extract information about the bug, which may require expert knowledge. Can LLMs help reduce

the requirement for expert knowledge? The road rock for this aim may lie in the limitation of input tokens for LLMs. How can we resolve this challenge?

5) *For Fuzzing*: While existing research demonstrates that LLMs perform better than traditional solutions for generating fuzz drivers, LLMs might still have the limitations of generating corner test cases. However, bugs may exist in those corner cases. Therefore, future works might explore the potential for LLMs to generate corner fuzz drivers for targets. Input generation and mutation for fuzzing have also been investigated in existing works. However, they ignore the fact that execution speed is critical for fuzzing because higher execution speed provides fuzzing with more chances to test targets. Therefore, future works might investigate how to improve execution speed when using LLMs. For example, one solution might be the compression of LLMs based on knowledge distillation.

There are other challenges impacting the performance of defect detection in fuzzing. When developers try to get feedback from software, instrumentation is a frequently used method. However, instrumentation is not a free lunch, and it requires understanding of instrumentation technologies, fuzzing, and targets. Many beginners spend much time on understanding the concepts in instrumentation and fuzzing. Therefore, LLMs can be used to help developers understand those concepts or even help instrument target programs. Similar to instrumentation, the execution environment of targets also requires expertise. For example, if fuzzing tries to test hypervisors, LLMs can help build the environment of running hypervisors. As for the monitor of defects, fuzzing also struggles to detect bugs that do not manifest themselves as crashes. Such bugs will remain silent even when they are triggered. Can LLMs help catch those bugs at runtime? Finally, to speed up the process of defect discovery, it might be possible to use LLMs to predict the bug-prone code regions while guiding fuzzing towards those regions. This intends to optimize the process of seed selection, byte selection, or byte mutation.

6) *For Unit Test*: As to unit test, current use of LLMs involves generating code snippets and inserting them into a test harness. However, the construction of a test harness still requires many manual efforts. Can LLMs help generate those test harnesses? While LLMs can already generate oracles for unit tests, how can we improve the quality of LLM-generated oracles? In order to have high-quality unit test, how can LLMs help in selecting the units?

7) *For Bug Analysis*: For program repair, current works focus on behavioral repair, i.e., fixing programs by patching code. Can LLMs help with state repair, i.e., fixing programs by correcting runtime states, such as memory size or execution time? To successfully fix a bug, current use of LLMs usually needs several attempts. How to improve the rate of successfully fixing programs in a one-time trial?

8) *Evaluation on LLM Assisted Techniques*: LLMs have already demonstrated their potential in enhancing software security techniques. However, there is a growing need for research to evaluate and compare different approaches to fully explore the possibilities of using LLMs in this domain. Currently, the focus of LLM usage in software security is primar-

ily on zero-shot and few-shot learning, with some emerging interest in the power of fine-tuning and pre-training LLMs. The key research question is: how effectively does each approach perform in the context of software security? The result of such research will guide future use of LLMs in an efficient and effective way.

B. Extensions From Existing Deep Learning Research

Deep learning has been used in many topics of software security [10], [16]. As to the future directions, we mainly discuss the topics of evaluation corpus, defect prediction, bug localization, closed-source bug detection, and code clone detection. There are many other research topics that use deep learning on software security, such as malware detection [121], security-related communication detection in Android applications [122], and symbolic execution [31]. These areas can also be explored to use LLMs.

1) *Generation of Evaluation Corpus*: In the field of software security, evaluating techniques based on a corpus is a common practice [48], [123]. Some works evaluate their proposed technique on a real-world corpus that is manually collected [123], while some others evaluate on synthesized corpus [48]. There is a debate about whether synthesized data is representative enough to be used in evaluating techniques [124]. Traditionally, the synthetic corpus has long been criticized for its unreality and thus, such data cannot reflect the real ability of techniques. Due to the capability of LLMs, it might have the potential to generate real enough corpora for the evaluation of techniques [118]. Recently, some studies have focused on the security or function correctness of code generated by LLMs [125]. They intend to fix the security issues of code generated by LLMs. However, if we utilize this ability to generate buggy code, we can synthesize corpora for evaluating techniques in software security. The research question is, how can we guide LLMs to generate the buggy code that we want? For example, can we use LLMs to generate concurrent bugs that exist in protocol programs? Another question is, how to prove that the buggy code generated by LLMs is representative enough for evaluating techniques? This question implies another question that is, how do we define the realness of a corpus? Overall, the aim of evaluation is to demonstrate the usefulness of a technique in real-world scenarios.

2) *Defect Prediction*: Defect prediction is very similar to data-driven bug detection, both of which extract features of defects to achieve their goals. The slight difference is that bug detection tries to determine the existence of bugs while defect prediction endeavors to anticipate the likelihood of certain code regions being defect-prone [126]. During the development of a project, defect prediction helps testers narrow down the target regions to be tested. Since deep learning has already been used in defect prediction, it is natural to use LLMs in this area. We can anticipate an increase in performance when using LLMs in defect prediction. Yet, the problem may be the dataset to fine-tune LLMs. As mentioned in Section IV-C, the performance of LLMs is not well when detecting bugs in a zero-shot manner. This will predominantly hold true for defect prediction as well, given the substantial commonalities they

share. Therefore, data collection might be the major road rock for using LLMs in defect prediction.

3) *Bug Localization*: As mentioned in Section IV-A, bug localization is the basis of automatic program repair. Bug localization does not only determine the existence of bugs but also pinpoint the locations of buggy code lines. To automate the process of bug localization, deep learning is utilized in this field [127]. Similar to other fields of using deep learning, the lack of data has long been an obstacle to improving the performance of deep learning in bug localization. Compared to bug detection, bug localization necessitates training data that encapsulates more comprehensive bug information, such as buggy code lines. This will also be an obstacle when using LLMs in bug localization. To explore more possibilities of LLMs, it is interesting to test their ability of bug localization. As studied by Huo *et al.*, transfer learning can mitigate the problem of lacking data by transferring knowledge from one buggy dataset to another [127]. Therefore, it may have great potential to fine-tune an LLM, which mostly uses transfer learning, for bug localization.

4) *Bug Detection in Closed-Source or Binary Code*: When pre-training an LLM, the large-scale training data probably contains open-source code. However, for closed-source code, LLMs might not recognize them well due to the lack of closed-source training data. Deep learning has already been used in detecting bugs in closed-source applications [128]. These works mainly use the similarity between the buggy code and the target code to determine if bugs exist in closed-source code [128]. With the LLM's ability of understanding source code, LLMs might also contain representations that help understand binary code. To detect bugs in binary code, one possible method is to fine-tune LLMs based on binary buggy code. This might need a large dataset of binary buggy code because one source code snippet can be compiled into many different versions of binary code, depending on the compilation options, platform architectures (e.g., x86, ARM or MIPS), and even compilers. Another method is to first use LLMs to translate binary code into source code or intermediate representation based on reverse engineering, and then use LLMs to detect bugs on the reversed code. Existing research has already explored the ability of LLMs in reverse engineering [129], which highlights the potential of using LLMs in bug detection in closed-source code. This direction can help mitigate security issues in applications such as IoT (Internet of Things) firmware and COTS (commercial off-the-shelf) applications.

5) *Code Clone Detection*: Code clone can introduce security issues in software [130]. To improve the speed of software development, developers may copy code to different locations directly or with small modifications, i.e., code clone. If the code snippet being copied is vulnerable, other copies are also vulnerable. Therefore, code clone detection can also help improve the efficiency of bug detection or bug prediction. To detect code clone in projects, deep learning is used to mine the semantic meaning and representations of code snippets [131]. Therefore, it also has great potential to use LLMs for code clone detection. In the field of software security, LLMs might be able to detect code clone as well as bugs at the same time.

Due to the limited length of input tokens of LLMs, it might be challenging to detect code clone in large-scale code. Future research may work on such issues.

VI. CONCLUSION

In this survey, we systematically review the results of using LLMs in software security. We conclude the methods of using LLMs for fuzzing, unit test, program repair, bug reproduction, data-driven bug detection, and bug triage. We analyze the technique details of each method, and discover that most existing works use LLMs in three steps, i.e., pre-processing, prompt generation, and post-processing. Finally, we discuss the future directions of using LLMs in software security from two aspects, including the possible improvement of the existing use of LLMs and the possible use of LLMs based on the works in traditional deep learning.

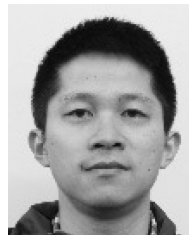
REFERENCES

- [1] H. Shahriar and M. Zulkernine, "Mitigating program security vulnerabilities: Approaches and challenges," *ACM Comput. Surv.*, vol. 44, no. 3, p. 11, Jun. 2012.
- [2] P. Achimugu, A. Selamat, R. Ibrahim, and M. N. Mahrin, "A systematic literature review of software requirements prioritization research," *Inf. Software Technol.*, vol. 56, no. 6, pp. 568–585, Jun. 2014.
- [3] C. Zhang and D. Budgen, "What do we know about the effectiveness of software design patterns?," *IEEE Trans. Software Eng.*, vol. 38, no. 5, pp. 1213–1231, Sep.–Oct. 2012.
- [4] M. Jorgensen and M. Shepperd, "A systematic review of software development cost estimation studies," *IEEE Trans. Software Eng.*, vol. 33, no. 1, pp. 33–53, Jan. 2007.
- [5] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11s, p. 230, Sep. 2022.
- [6] J. K. Siow, C. Gao, L. Fan, S. Chen, and Y. Liu, "CORE: Automating review recommendation for code changes," in *Proc. IEEE 27th Int. Conf. Software Analysis, Evolution and Reengineering*, London, Canada, 2020, pp. 284–295.
- [7] 2024 *Must-Know Cyber Attack Statistics and Trends*. [Online]. Available: <https://www.embroker.com/blog/cyber-attack-statistics>. Accessed on: Sep. 2024.
- [8] J. J. Gutiérrez, M. Escalona, and M. Mejias, "A Model-Driven approach for functional test case generation," *J. Syst. Software*, vol. 109, pp. 214–228, Nov. 2015.
- [9] K. Zhang, X. Xiao, X. Zhu, R. Sun, M. Xue, and S. Wen, "Path transitions tell more: Optimizing fuzzing schedules via runtime program states," in *Proc. 44th Int. Conf. Software Engineering*, Pittsburgh, USA, 2022, pp. 1658–1668.
- [10] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.
- [11] Y. Wei, Y. Pei, C. A. Furia, L. S. Silva, S. Buchholz, B. Meyer, and A. Zeller, "Automated fixing of programs with contracts," in *Proc. 19th Int. Symp. Software Testing and Analysis*, Trento, Italy, 2010, pp. 61–72.
- [12] Y. Zhao, T. Yu, T. Su, Y. Liu, W. Zheng, J. Zhang, and W. G. J. Halfond, "ReCDroid: Automatically reproducing android application crashes from bug reports," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering*, Montreal, Canada, 2019, pp. 128–139.
- [13] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. 25th Annu. Network and Distributed System Security Symp.*, San Diego, USA, 2018.
- [14] I. Alazzam, A. Aleroud, Z. Al Latifah, and G. Karabatis, "Automatic bug triage in software systems using graph neighborhood relations for feature augmentation," *IEEE Trans. Comput. Soc. Syst.*, vol. 7, no. 5, pp. 1288–1303, Oct. 2020.
- [15] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [16] Y. Yang, X. Xia, D. Lo, and J. Grundy, "A survey on deep learning for software engineering," *ACM Comput. Surv.*, vol. 54, no. 10s, p. 206, Sep. 2022.
- [17] P. Godefroid, H. Peleg, and R. Singh, "Learn&Fuzz: Machine learning for input fuzzing," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Software Engineering*, Urbana, USA, 2017, pp. 50–59.
- [18] N. Jiang, T. Lutellier, and L. Tan, "CURE: Code-aware neural machine translation for automatic program repair," in *Proc. IEEE/ACM 43rd Int. Conf. Software Engineering*, Madrid, Spain, 2021, pp. 1161–1173.
- [19] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," arXiv preprint arXiv: 2303.18223, 2023.
- [20] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proc. 32nd ACM SIGSOFT Int. Symp. Software Testing and Analysis*, Seattle, USA, 2023.
- [21] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Security and Privacy*, San Francisco, USA, 2023, pp. 1–18.
- [22] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint European Software Engineering Conf. Symp. Foundations of Software Engineering*, Singapore, Singapore, 2022, pp. 959–971.
- [23] J. Zhang, L. Pan, Q.-L. Han, C. Chen, S. Wen, and Y. Xiang, "Deep learning based attack detection for cyber-physical system cybersecurity: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 3, pp. 377–391, Mar. 2022.
- [24] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," arXiv preprint arXiv: 2405.04760, 2024.
- [25] T. Wu, S. He, J. Liu, S. Sun, K. Liu, Q.-L. Han, and Y. Tang, "A brief overview of ChatGPT: The history, status quo and potential future development," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 5, pp. 1122–1136, May 2023.
- [26] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Software Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.
- [27] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [28] R. Natella, D. Cotroneo, J. A. Duraes, and H. S. Madeira, "On fault representativeness of software fault injection," *IEEE Trans. Software Eng.*, vol. 39, no. 1, pp. 80–96, Jan. 2013.
- [29] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Comput. Surv.*, vol. 51, no. 3, p. 50, May 2018.
- [30] X. Liu, X. Li, R. Prajapati, and D. Wu, "DeepFuzz: Automatic generation of syntax valid c programs for fuzz testing," in *Proc. 33rd AAAI Conf. Artificial Intelligence*, Honolulu, USA, 2019, pp. 1044–1051.
- [31] J. Wen, T. Mahmud, M. Che, Y. Yan, and G. Yang, "Intelligent constraint classification for symbolic execution," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering*, Taipa, China, 2023, pp. 144–154.
- [32] OpenAI, "Introducing ChatGPT," [Online]. Available: <https://openai.com/blog/chatgpt>. Accessed on: Nov. 30, 2022.
- [33] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A.

- Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, “Code llama: Open foundation models for code,” arXiv preprint arXiv: 2308.12950, 2023.
- [34] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, “Emergent abilities of large language models,” arXiv preprint arXiv: 2206.07682, 2022.
- [35] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Kureger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” *Proc. 34th Int. Conf. Neural Information Processing Systems*, Vancouver, Canada, 2020, pp. 1877–1901.
- [36] N. Houlsby, A. Giurui, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for NLP,” in *Proc. 36th Int. Conf. Machine Learning*, Long Beach, USA, 2019, pp. 2790–2799.
- [37] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” in *Proc. 59th Annu. Meeting of the Association for Computational Linguistics and the 11th Int. Joint Conf. Natural Language Processing*, 2021, pp. 4582–4597.
- [38] J. Lee, K. Han, and H. Yu, “A light bug triage framework for applying large pre-trained language model,” in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering*, Rochester, USA, 2022, pp. 3.
- [39] G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, X. Sun, L. Bian, H. Wang, and Z. Wang, “Automated conformance testing for JavaScript engines via deep compiler fuzzing,” in *Proc. 42nd ACM SIGPLAN Int. Conf. Programming Language Design and Implementation*, Canada, 2021, pp. 435–450.
- [40] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, “Transformer-based language models for software vulnerability detection,” in *Proc. 38th Annu. Computer Security Applications Conf.*, Austin, USA, 2022, pp. 481–496.
- [41] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [42] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, “Fill in the blank: Context-aware automated text input generation for mobile GUI testing,” in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [43] N. Jiang, K. Liu, T. Lutellier, and L. Tan, “Impact of code language models on automated program repair,” in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [44] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [45] V. H. S. Durelli, R. S. Durelli, S. S. Borges, A. T. Endo, M. M. Eler, D. R. C. Dias, and M. P. Guimarães, “Machine learning applied to software testing: A systematic mapping study,” *IEEE Trans. Rel.*, vol. 68, no. 3, pp. 1189–1212, Sep. 2019.
- [46] X. Zhu and M. Böhme, “Regression greybox fuzzing,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security*, Republic of Korea, 2021, pp. 2169–2182.
- [47] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, “Detecting vulnerability on IoT device firmware: A survey,” *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [48] X. Zhu, X. Feng, T. Jiao, S. Wen, Y. Xiang, S. Camtepe, and J. Xue, “A feature-oriented corpus for understanding, evaluating and improving fuzz testing,” in *Proc. ACM Asia Conf. Computer and Communications Security*, Auckland, New Zealand, 2019, pp. 658–663.
- [49] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Trans. Software Eng.*, vol. 41, no. 5, pp. 507–525, May 2015.
- [50] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Trans. Software Engineering*, vol. 37, no. 5, pp. 649–678, Sep.–Oct. 2011.
- [51] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *Openai Blog*, vol. 1, no. 8, p. 9, 2019.
- [52] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, “Evaluating large language models trained on code,” arXiv preprint arXiv: 2107.03374, 2021.
- [53] B. Deka, Z. Huang, C. Franzen, J. Hibsman, D. Afergan, Y. Li, J. Nichols, and R. Kumar, “Rico: A mobile app dataset for building data-driven design applications,” in *Proc. 30th Annu. ACM Symp. User Interface Software and Technology*, Québec City, Canada, 2017, pp. 845–854.
- [54] K. Kuznetsov, C. Fu, S. Gao, D. N. Jansen, L. Zhang, and A. Zeller, “Frontmatter: Mining Android user interfaces at scale,” in *Proc. 29th ACM Joint Meeting on European Software Engineering Conf. Symp. Foundations of Software Engineering*, Athens, Greece, 2021, pp. 1580–1584.
- [55] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-T. Yih, L. Zettlemoyer, and M. Lewis, “InCoder: A generative model for code infilling and synthesis,” arXiv preprint arXiv: 2204.05999, 2022.
- [56] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, “Large language model guided protocol fuzzing,” in *Proc. 31st Annu. Network and Distributed System Security Symp.*, San Diego, USA, 2024.
- [57] P. Bareiß, B. Souza, M. d’Amorim, and M. Pradel, “Code generation tools (almost) for free? A study of few-shot, pre-trained language models on code,” arXiv preprint arXiv: 2206.01335, 2022.
- [58] Z. Xie, Y. Chen, C. Zhi, S. Deng, and J. Yin, “ChatUniTest: A ChatGPT-based automated unit test generation tool,” arXiv preprint arXiv: 2305.04764v1, 2023.
- [59] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Trans. Software Eng.*, vol. 50, no. 1, pp. 85–105, Jan. 2024.
- [60] N. Rao, K. Jain, U. Alon, C. Le Goues, and V. J. Hellendoorn, “CAT-LM training language models on aligned code and tests,” in *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering*, Echternach, Luxembourg, 2023, pp. 409–420.
- [61] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, and Y. Xiang, “Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security*, Republic of Korea, 2021, pp. 337–350.
- [62] X. Zhu, X. Feng, X. Meng, S. Wen, S. Camtepe, Y. Xiang, and K. Ren, “CSI-Fuzz: Full-speed edge tracing using coverage sensitive instrumentation,” *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 912–923, Mar.–Apr. 2022.
- [63] D. Babić, S. Bucur, Y. Chen, F. Ivančić, T. King, M. Kusano, C. Lemieux, L. Szekeres, and W. Wang, “FUDGE: Fuzz driver generation at scale,” in *Proc. 27th ACM Joint Meeting on European Software Engineering Conf. Symp. Foundations of Software Engineering*, Tallinn, Estonia, 2019, pp. 975–985.
- [64] N. Jain, S. Vaidyanath, A. Iyer, N. Natarajan, S. Parthasarathy, S. Rajamani, and R. Sharma, “Jigsaw: Large language models meet program synthesis,” in *Proc. IEEE/ACM 44th Int. Conf. Software Engineering*, Pittsburgh, USA, 2022, pp. 1219–1231.
- [65] H. Strobel, A. Webson, V. Sanh, B. Hoover, J. Beyer, H. Pfister, and A. M. Rush, “Interactive and visual prompt engineering for ad-hoc task adaptation with large language models,” *IEEE Trans. Vis. Comput. Graph.*, vol. 29, no. 1, pp. 1146–1156, Jan. 2023.
- [66] B. Jeong, J. Jang, H. Yi, J. Moon, J. Kim, I. Jeon, T. Kim, W. C. Shim, and Y. H. Hwang, “UTOPIA: Automatic generation of fuzz driver using unit tests,” in *Proc. IEEE Symp. Security and Privacy*, San Francisco, USA, 2023, pp. 746–762.

- [67] L. Williams, G. Kudrjavets, and N. Nagappan, "On the effectiveness of unit test automation at microsoft," in *Proc. 20th Int. Symp. Software Reliability Engineering*, Mysuru, India, 2009, pp. 81–89.
- [68] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. 19th ACM SIGSOFT Symp. 13th European Conf. Foundations of Software Engineering*, Szeged, Hungary, 2011, pp. 416–419.
- [69] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for Java," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems and Applications Companion*, Montreal, Canada, 2007, pp. 815–816.
- [70] M. M. Almasi, H. Hemmati, G. Fraser, A. Arcuri, and J. Benefelds, "An industrial evaluation of unit test generation: Finding real faults in a financial application," in *Proc. IEEE/ACM 39th Int. Conf. Software Engineering: Software Engineering in Practice Track*, Buenos Aires, Argentina, 2017, pp. 263–272.
- [71] F. E. Allen, "Control flow analysis," *ACM SIGPLAN Not.*, vol. 5, no. 7, pp. 1–19, Jul. 1970.
- [72] L. D. Fosdick and L. J. Osterweil, "Data flow analysis in software reliability," *ACM Comput. Surv.*, vol. 8, no. 3, pp. 305–330, Sep. 1976.
- [73] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering*, Montreal, Canada, 2019, pp. 783–794.
- [74] J. A. Prenner and R. Robbes, "Automatic program repair with OpenAI's codex: Evaluating QuixBugs," arXiv preprint arXiv: 2111.03922, 2021.
- [75] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. Findings of the Association for Computational Linguistics*, 2020, pp. 1536–1547.
- [76] J. Zhang, J. Cambronero, S. Gulwani, V. Le, R. Piskac, G. Soares, and G. Verbruggen, "Repairing bugs in python assignments using large language models," arXiv preprint arXiv: 2209.14876, 2022.
- [77] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "VulRepair: A t5-based automated software vulnerability repair," in *Proc. 30th ACM Joint European Software Engineering Conf. Symp. Foundations of Software Engineering*, Singapore, Singapore, 2022, pp. 935–947.
- [78] N. Tihanyi, R. Jain, Y. Charalambous, M. A. Ferrag, Y. Sun, and L. C. Cordeiro, "A new era in software security: Towards self-healing software via large language models and formal verification," arXiv preprint arXiv: 2305.14752, 2023.
- [79] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods in Natural Language Processing*, Punta Cana, Dominican Republic, 2021, pp. 8696–8708.
- [80] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering*, Melbourne, Australia, 2023.
- [81] AI21, "Announcing ai21 studio and jurassic-1 language models," [Online]. Available: <https://www.ai21.com/blog/announcing-ai21-studio-and-jurassic-1>. Accessed on: Jul., 2023.
- [82] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proc. 6th ACM SIGPLAN Int. Symp. Machine Programming*, San Diego, USA, 2022, pp. 1–10.
- [83] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 2655–2668.
- [84] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An open large language model for code with multi-turn program synthesis," in *Proc. 11th Int. Conf. Learning Representations*, Kigali, Rwanda, 2023.
- [85] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, and S. Shah, "How effective are neural networks for fixing security vulnerabilities," in *Proc. 32nd ACM SIGSOFT Int. Symp. Software Testing and Analysis*, Seattle, USA, 2023.
- [86] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering*, Melbourne, Australia, 2023, pp. 2450–2462.
- [87] K. Huang, X. Meng, J. Zhang, Y. Liu, W. Wang, S. Li, and Y. Zhang, "An empirical study on fine-tuning large language models of code for automated program repair," in *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering*, Luxembourg, Luxembourg, 2023, pp. 1162–1174.
- [88] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphcodeBERT: Pre-training code representations with data flow," in *Proc. 9th Int. Conf. Learning Representations*, Austria, 2021.
- [89] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," in *Proc. 31st ACM Joint European Software Engineering Conf. Symp. Foundations of Software Engineering*, San Francisco, USA, 2023, pp. 172–184.
- [90] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-Lm: Training multi-billion parameter language models using model parallelism," arXiv preprint arXiv: 1909.08053, 2019.
- [91] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv: 1907.11692, 2019.
- [92] M. R. Taesiri, F. Macklon, Y. Wang, H. Shen, and C.-P. Bezemer, "Large language models are pretty good zero-shot video game bug detectors," arXiv preprint arXiv: 2210.02506, 2022.
- [93] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. 36th Int. Conf. Neural Information Processing Systems*, New Orleans, USA, 2022, pp. 2011.
- [94] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: Open pre-trained transformer language models," arXiv preprint arXiv: 2205.01068, 2022.
- [95] A. Cheshkov, P. Zadorozhny, and R. Levichev, "Evaluation of ChatGPT model for vulnerability detection," arXiv preprint arXiv: 2304.07232, 2023.
- [96] M. Monperrus, "Automatic software repair: A bibliography," *ACM Comput. Surv.*, vol. 51, no. 1, p. 17, Jan. 2019.
- [97] H. Zhong and Z. Su, "An empirical study on real bug fixes," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Software Engineering*, Florence, Italy, 2015, pp. 913–923.
- [98] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Trans. Software Eng.*, vol. 42, no. 8, pp. 707–740, Aug. 2016.
- [99] S. D. Kolak, R. Martins, C. Le Goues, and V. J. Hellendoorn, "Patch generation with language models: Feasibility and scaling behavior," in *Proc. Deep Learning for Code Workshop*, 2022.
- [100] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "Fixing hardware security bugs with large language models," arXiv preprint arXiv: 2302.01215, 2023.
- [101] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "QuixBugs: A multi-lingual program repair benchmark set based on the quixey challenge," in *Proc. ACM SIGPLAN Int. Conf. Systems, Programming, Languages, and Applications: Software for Humanity*, Vancouver, Canada, 2017, pp. 55–56.
- [102] "Common weakness enumeration: A community-developed list of software and hardware weakness types," [Online]. Available: <https://cwe.mitre.org/>. Accessed on: Jul., 2023.
- [103] A. Biere, A. Cimatti, E. M. Clarke, O. Strichman, and Y. Zhu, "Bounded model checking," *Handbook Satisfiability*, vol. 185, no. 99,

- pp. 457–481, Feb. 2009.
- [104] M. Gordon, “The semantic challenge of Verilog HDL,” in *Proc. 10th Annu. IEEE Symp. Logic in Computer Science*, San Deigo, USA, 1995, pp. 136–145.
- [105] M. Fazzini, M. Prammer, M. d’Amorim, and A. Orso, “Automatically translating bug reports into test cases for mobile apps,” in *Proc. 27th ACM SIGSOFT Int. Symp. Software Testing and Analysis*, Amsterdam, Netherlands, 2018, pp. 141–152.
- [106] N. Chen and S. Kim, “STAR: Stack trace based automatic crash reproduction via symbolic execution,” *IEEE Trans. Software Eng.*, vol. 41, no. 2, pp. 198–220, Feb. 2015.
- [107] S. Artzi, S. Kim, and M. D. Ernst, “ReCrash: Making software failures reproducible by preserving object states,” in *Proc. 22nd European Conf. Paphos*, Cyprus, 2008, pp. 542–565.
- [108] W. Jin and A. Orso, “BugRedux: Reproducing field failures for in-house debugging,” in *Proc. 34th Int. Conf. Software Engineering*, Zurich, Switzerland, 2012, pp. 474–484.
- [109] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, “SySeVR: A framework for using deep learning to detect software vulnerabilities,” *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2244–2258, 2022.
- [110] G. Grieco, G. L. Grinblat, L. Uzal, S. Rawat, J. Feist, and L. Mounier, “Toward large-scale vulnerability discovery using machine learning,” in *Proc. 6th ACM Conf. Data and Application Security and Privacy*, New Orleans, USA, 2016, pp. 85–96.
- [111] T. Le, T. V. Nguyen, T. Le, D. Phung, P. Montague, O. De Vel, and L. Qu, “Maximal divergence sequential auto-encoder for binary software vulnerability detection,” in *Proc. Int. Conf. Learning Representations*, La Jolla, USA, 2019.
- [112] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, “Automated vulnerability detection in source code using deep representation learning,” in *Proc. 17th IEEE Int. Conf. Machine Learning and Applications*, Orlando, USA, 2018, pp. 757–762.
- [113] J. Xuan, H. Jiang, Y. Hu, Z. Ren, W. Zou, Z. Luo, and X. Wu, “Towards effective bug triage with software data reduction techniques,” *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 1, pp. 264–280, Jan. 2015.
- [114] S. Mani, A. Sankaran, and R. Aralikatte, “DeepTriage: Exploring the effectiveness of deep learning for bug triaging,” in *Proc. ACM India Joint Int. Conf. Data Science and Management of Data*, Kolkata, India, 2019, pp. 171–179.
- [115] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” arXiv preprint arXiv: 1503.02531, 2015.
- [116] N. Zeng, P. Wu, Z. Wang, H. Li, W. Liu, and X. Liu, “A small-sized object detection oriented multi-scale feature fusion approach with application to defect detection,” *IEEE Trans. Instrum. Meas.*, vol. 71, p. 3507014, Feb. 2022.
- [117] N. Zeng, P. Wu, Y. Zhang, H. Li, J. Mao, and Z. Wang, “DPSMN: A dual-pathway multiscale network for image forgery detection,” *IEEE Trans. Industr. Inform.*, vol. 20, no. 5, pp. 7665–7674, May 2024.
- [118] K. M. Yoo, D. Park, J. Kang, S.-W. Lee, and W. Park, “GPT3Mix: Leveraging large-scale language models for text augmentation,” in *Proc. Findings of the Association for Computational Linguistics*, Punta Cana, Dominican Republic, 2021, pp. 2225–2239.
- [119] L. Bonifacio, H. Abonizio, M. Fadaee, and R. Nogueira, “InPars: Data augmentation for information retrieval using large language models,” arXiv preprint arXiv: 2202.05144, 2022.
- [120] G. Sahu, P. Rodriguez, I. Laradji, P. Atighehchian, D. Vazquez, and D. Bahdanau, “Data augmentation for intent classification with off-the-shelf large language models,” in *Proc. 4th Workshop on NLP for Conversational AI*, Dublin, Ireland, 2022.
- [121] X. Chen, C. Li, D. Wang, S. Wen, J. Zhang, S. Nepal, Y. Xiang, and K. Ren, “Android HIV: A study of repackaging malware for evading machine-learning detection,” *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 987–1001, Jul. 2020.
- [122] J. Zhao, A. Albarghouthi, V. Rastogi, S. Jha, and D. Octeau, “Neural-augmented static analysis of android communication,” in *Proc. 26th ACM Joint Meeting on European Software Engineering Conf. Symp. Foundations of Software Engineering*, Lake Buena Vista, USA, 2018, pp. 342–353.
- [123] Y. Li, S. Ji, Y. Chen, S. Liang, W.-H. Lee, Y. Chen, C. Lyu, C. Wu, R. Beyah, P. Cheng, K. Lu, and T. Wang, “UNIFUZZ: A holistic and pragmatic Metrics-Driven platform for evaluating fuzzers,” in *Proc. 30th USENIX Security Symp.*, 2021, pp. 2777–2794.
- [124] M. Bohme, C. Cadar, and A. Roychoudhury, “Fuzzing: Challenges and reflections,” *IEEE Software*, vol. 38, no. 3, pp. 79–86, May-Jun. 2021.
- [125] K. Jesse, T. Ahmed, P. T. Devanbu, and E. Morgan, “Large language models and simple, stupid bugs,” in *Proc. IEEE/ACM 20th Int. Conf. Mining Software Repositories*, Melbourne, Australia, 2023.
- [126] S. Wang, T. Liu, and L. Tan, “Automatically learning semantic features for defect prediction,” in *Proc. 38th Int. Conf. Software Engineering*, Austin, USA, 2016, pp. 297–308.
- [127] X. Huo, F. Thung, M. Li, D. Lo, and S.-T. Shi, “Deep transfer bug localization,” *IEEE Trans. Software Eng.*, vol. 47, no. 7, pp. 1368–1380, Jul. 2021.
- [128] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security*, Dallas, USA, 2017, pp. 363–376.
- [129] H. Pearce, B. Tan, P. Krishnamurthy, F. Khorrami, R. Karri, and B. Dolan-Gavitt, “Pop quiz! Can a large language model help with reverse engineering?” arXiv preprint arXiv: 2202.01142, 2022.
- [130] S. Kim, S. Woo, H. Lee, and H. Oh, “VUDDY: A scalable approach for vulnerable code clone discovery,” in *Proc. IEEE Symp. Security and Privacy*, San Jose, USA, 2017, pp. 595–614.
- [131] M. White, M. Tufano, C. Vendome, and D. Poshvyanyk, “Deep learning code fragments for code clone detection,” in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering*, Singapore, Singapore, 2016, pp. 87–98.



Xiaogang Zhu (Member, IEEE) received the Ph.D. degree in computer science and engineering from Swinburne University of Technology. Currently, he is a Lecturer at the University of Adelaide and focuses on searching vulnerabilities in programs. He is interested in detecting techniques such as fuzzing, machine learning and symbolic execution. He has published papers on top journals, such as *TDSC*, and conferences such as CCS, USENIX Security, and ICSE. He also served as a Reviewer for many top journals such as *TDSC*, *IoTJ*, and *CSUR*.



Wei Zhou received the B.Eng. and M.Eng. degrees from Central South University, in 2005 and 2008, respectively, all in CS, and the Ph.D. degree from School of Engineering and IT, University of New South Wales, Australia, in 2016. Her research interests include computer networks, network security, and fingerprint biometric.



Qing-Long Han (Fellow, IEEE) received the B.Sc. degree in Mathematics from Shandong Normal University, Jinan, China, in 1983, and the M.Sc. and Ph.D. degrees in Control Engineering from East China University of Science and Technology, Shanghai, China, in 1992 and 1997, respectively.

Professor Han is Pro Vice-Chancellor (Research Quality) and a Distinguished Professor at Swinburne University of Technology, Melbourne, Australia. He held various academic and management positions at Griffith University and Central Queensland University, Australia. His research interests include networked control systems, multi-agent systems, time-delay systems, smart grids, unmanned surface vehicles, and neural networks.

Professor Han was awarded the 2024 IEEE Dr.-Ing. Eugene Mittelmann Achievement Award (the Highest Award in Industrial Electronics), the 2021

Norbert Wiener Award (the Highest Award in systems science and engineering, and cybernetics) and the 2021 M. A. Sargent Medal (the Highest Award of the Electrical College Board of Engineers Australia).

He was the recipient of the IEEE Systems, Man, and Cybernetics Society Andrew P. Sage Best Transactions Paper Award in 2022, 2020, and 2019, respectively, the IEEE/CAA Journal of Automatica Sinica Norbert Wiener Review Award in 2020, and the IEEE Transactions on Industrial Informatics Outstanding Paper Award in 2020.

Professor Han is a Member of the Academia Europaea (The Academy of Europe). He is a Fellow of The International Federation of Automatic Control (FIFAC), an Honorary Fellow of The Institution of Engineers Australia (HonFIEAust), and a Fellow of Chinese Association of Automation (FCAA). He is a Highly Cited Researcher in both Engineering and Computer Science (Clarivate). He has served as an AdCom Member of IEEE Industrial Electronics Society (IES), a Member of IEEE IES Fellows Committee, a Member of IEEE IES Publications Committee, Chair of IEEE IES Technical Committee on Network-Based Control Systems and Applications, and the Co-Editor-in-Chief of IEEE Transactions on Industrial Informatics. He is currently the Editor-in-Chief of IEEE/CAA Journal of Automatica Sinica and the Co-Editor of Australian Journal of Electrical and Electronic Engineering.



Wanlun Ma (Member, IEEE) received the bachelor and master degrees in information and communication engineering from the University of Electronic Science and Technology of China (UESTC), in 2017 and 2020, respectively; the Ph.D. degree in computer science from Swinburne University of Technology, Australia, in 2024. He is currently a Research Fellow with Swinburne University of Technology. His research interests focus on trustworthy and responsible AI, adversarial machine learning, and

network security and privacy.



Sheng Wen (Senior Member, IEEE) received the Ph.D. degree from Deakin University, Australia, in October 2014. Now, he is working full-time as an Associate Professor in Swinburne University of Technology. He managed several research projects in the last few years. He is now the Director of Blockchain Innovation Lab and the Deputy Director of Swinburne Cybersecurity Lab in Swinburne University. He is also leading a medium-sized research team with co-supervised Ph.D. students in the system security area. In addition, he has published over 100 high-quality papers, including top conference papers such as papers in IEEE S&P, ACM CCS, NDSS, ICSE and FSE, as well as many papers in IEEE/ACM transactions series journals.



Yang Xiang (Fellow, IEEE) received the Ph.D. degree in computer science from Deakin University, Australia. He is currently a Full Professor and the Dean of Digital Research, Swinburne University of Technology, Australia. His personal research interests include cyber security, which covers network and system security, data analytics, distributed systems, and networking. In the past 20 years, he has published more than 300 research papers in many international journals and conferences. He is the Editor-in-Chief of the *SpringerBriefs on Cyber Security Systems and Networks*. He serves as the Associate Editor of *IEEE Transactions on Dependable and Secure Computing*, *IEEE Internet of Things Journal*, and *ACM Computing Surveys*. He is the Coordinator, Asia for IEEE Computer Society Technical Committee on Distributed Processing (TCDP), and the Chair of the Australia and New Zealand, IEEE Blockchain Technical Community.